

Reviewer Pack — Minimal Rank of Noncrossing Partitions vs Communication Comp...

Entry 144390026133 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 04:44:53 UTC

Minimal Rank of Noncrossing Partitions vs Communication Complexity of Disjointness

Entry ID: 144390026133 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 04:44:53 UTC

1. Conjecture

Field A (mathematical branch): Noncrossing Partition Theory **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

[‘For any disjointness function $f: \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$, the minimal rank of the noncrossing partition matrix representation of f is at least $\Omega(n \log n)$.’, ‘This lower bound holds for all instances with $n \leq 40$.’, ‘The minimal rank of a noncrossing partition matrix represents the minimum number of parts into which the input domain can be partitioned such that no two elements from different parts have a noncrossing edge.’]

Rationale (proposer’s reasoning):

[‘Noncrossing partitions are known to encode combinatorial structures with certain graph properties, and their minimal rank has been studied in algebraic combinatorics. A connection between this structure and communication complexity of disjointness could reveal underlying complexity-theoretic principles.’, ‘The conjecture leverages the well-studied nature of noncrossing partitions to provide a potential lower bound on the communication complexity of disjointness, which is a challenging problem with connections to cryptography.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7caf53cd15ca3b75

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman rank correlation coefficient between the minimal rank of noncrossing partition matrices and communication complexity is ≥ 0.8 across all $n \leq 40$ disjointness functions, with a mean value ≤ 3 for any seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncrossing partition theory" AND "communication complexity" AND "disjointness" - "minimal rank" AND "noncrossing partition" AND " $O(n \log n)$ " - "partition matrix representation" AND "disjointness function" AND "lower bound"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
import math
import itertools

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_disjointness_function(n):
        return lambda x, y: sum(xi != yi for xi, yi in zip(x, y))

    def noncrossing_partition_matrix(f, n):
        A = [[f(i, j) for j in range(n)] for i in range(n)]
        rank = 0
        for k in range(1, n + 1):
            for partition in itertools.combinations(range(n), k):
                B = [A[i][j] for i in partition for j in partition]
                if is_independent(B):
                    rank += 1
        return rank

    def is_independent(matrix):
        m = len(matrix)
        n = len(matrix[0])
        A = matrix + [[0] * n for _ in range(m)]
        for i in range(m):
            pivot = next((j for j in range(n) if A[i][j]), None)
            if pivot is None:
                return False
            for j in range(i, m):
                for k in range(pivot, n):
                    A[j][k] -= A[i][k] * A[i][pivot]
```

```

        A[j][pivot] /= A[i][pivot]
    for j in range(m):
        if j != i and any(A[j][k] != 0 for k in range(n)):
            for k in range(n):
                A[j][k] -= A[i][k] * A[j][pivot]
    return True

def communication_complexity(f, n):
    # Example protocol:  $O(n)$  bits
    return n

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_disjointness_function(n)
    rank = noncrossing_partition_matrix(f, n)
    comm_complexity = communication_complexity(f, n)
    results.append((rank, comm_complexity))

if len(results) < 30:
    return {
        "metric_name": "Spearman rank correlation",
        "metric_value": None,
        "instances_tested": len(results),
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

ranks = [r for r, _ in results]
comm_complexities = [c for _, c in results]

def spearman_rank_correlation(ranks, comm_complexities):
    n = len(ranks)
    rank_ranks = {x: i + 1 for i, x in enumerate(sorted(set(ranks)))}
    rank_comm_complexities = {x: i + 1 for i, x in enumerate(sorted(set(comm_complexities)))}
    sum_diff_squares = sum((rank_ranks[r] - rank_comm_complexities[c]) ** 2 for r, c in zip(ranks, comm_complexities))
    return 1 - (6 * sum_diff_squares) / (n * (n**2 - 1))

correlation = spearman_rank_correlation(ranks, comm_complexities)
mean_metric_value = correlation
support_fraction = len([r for r in ranks if r >= 0.8]) / len(ranks)

return {
    "metric_name": "Spearman rank correlation",
    "metric_value": correlation,
    "instances_tested": len(results),
    "conjecture_holds": support_fraction >= 0.8 and mean_metric_value <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 7 for i in range(5, 8)]

    results = []

```

```

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result)

all_metric_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all_metric_values and support_fraction >= 0.8:
    mean_metric_value = sum(all_metric_values) / len(all_metric_values)
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=NA support_fraction={support_fraction}")
else:
    first_failing_seed = next((i for i, r in enumerate(results) if not r["conjecture_holds"]), None)
    counterexample = "insufficient_support" if not all_metric_values else ""
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f4627dd.py", line 99, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f4627dd.py", line 58, in run_trial
    rank = noncrossing_partition_matrix(f, n)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f4627dd.py", line 24, in noncrossing_partition_matrix
    A = [[f(i, j) for j in range(n)] for i in range(n)]
         ^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f4627dd.py", line 21, in <lambda>
    return lambda x, y: sum(xi != yi for xi, yi in zip(x, y))
                               ^^^^^^^^^
TypeError: 'int' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11459
2	preregistration	ollama_remote	glm4:latest	0	0	5454
3	novelty	ollama_remote	glm4:latest	0	0	4742
4	novelty	ollama_remote	glm4:latest	0	0	5311
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18532
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14776
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9368
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12209
9	judge	ollama_remote	glm4:latest	0	0	9408

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 91257 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/144390026133.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/144390026133.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/144390026133.tar.gz` (if generated)